

BGU CIS 2024 Version HPC Cluster User Guide

02/06/2026

Contents

Abstract.....	5
Use	7
Submitting a Job.....	8
Batch File.....	8
Allocating Resources	9
Running Thousands of Jobs.....	9
Interactive vs Non-Interactive Use	10
Information about the compute nodes	11
List of My Currently Running Jobs	11
Cancel Jobs.....	11
Cancel All Pending jobs for a Specific User	11
Running Job Information	11
Complete Job Information	11
Resources Usage	11
Advanced Topics	12
Jupyter Lab.....	12
Installation	12
Make the Conda Environment Available in Notebook's Interface	12
Launch Jupyter Lab	12
Release Job Resources from Within Jupyter After Code Has Finished Running.....	13
Tensorboard.....	13
Working with Notebooks	13
Usefull SBATCH variables	14
High Priority Jobs (Golden Tickets)	15
Prioritize Your Own Jobs	16
Allocate Extra RAM/CPU's	17
Working with the Compute Node's SSD Drive.....	18
Sending Arguments to sbatch File	19
Job Arrays.....	20
Send Name of an Input File to Each Task.....	20
Read a Line from an Input File for Each Task.....	20
Email Notifications	21

Limiting the Number of Simultaneously Running Tasks from the Job Array	21
Job Dependencies	22
CUDA Version Selection	23
IDEs	24
pyCharm	24
Visual Studio Code	27
Docker	31
Apptainer	31
UDOCKER.....	31
Matlab.....	34
julia.....	35
R	36
Command Line	36
R in Jupyter.....	36
R in MS Visual Studio Code	36
RStudio.....	37
C#	38
Install In Conda Environment.....	38
Use	38
Adding Packages to .Net	38
Adding Project to Solution	38
Fiji – Image Analysis Tool	39
Appendix	40
Step by Step Guide for First Use of Python and Conda	40
Example for Creating Latest Tensorflow-gpu and Jupyter Lab Environment	41
Conda	42
Viewing a list of your environments	42
list of all packages installed in a specific environment	42
Activating / deactivating environment	42
Create Environment	42
Remove Environment	43
Update Conda	43
Compare Conda Environments	43

Transfer Files 44

 To / From Your PC 44

 Get a Public File..... 44

Github 45

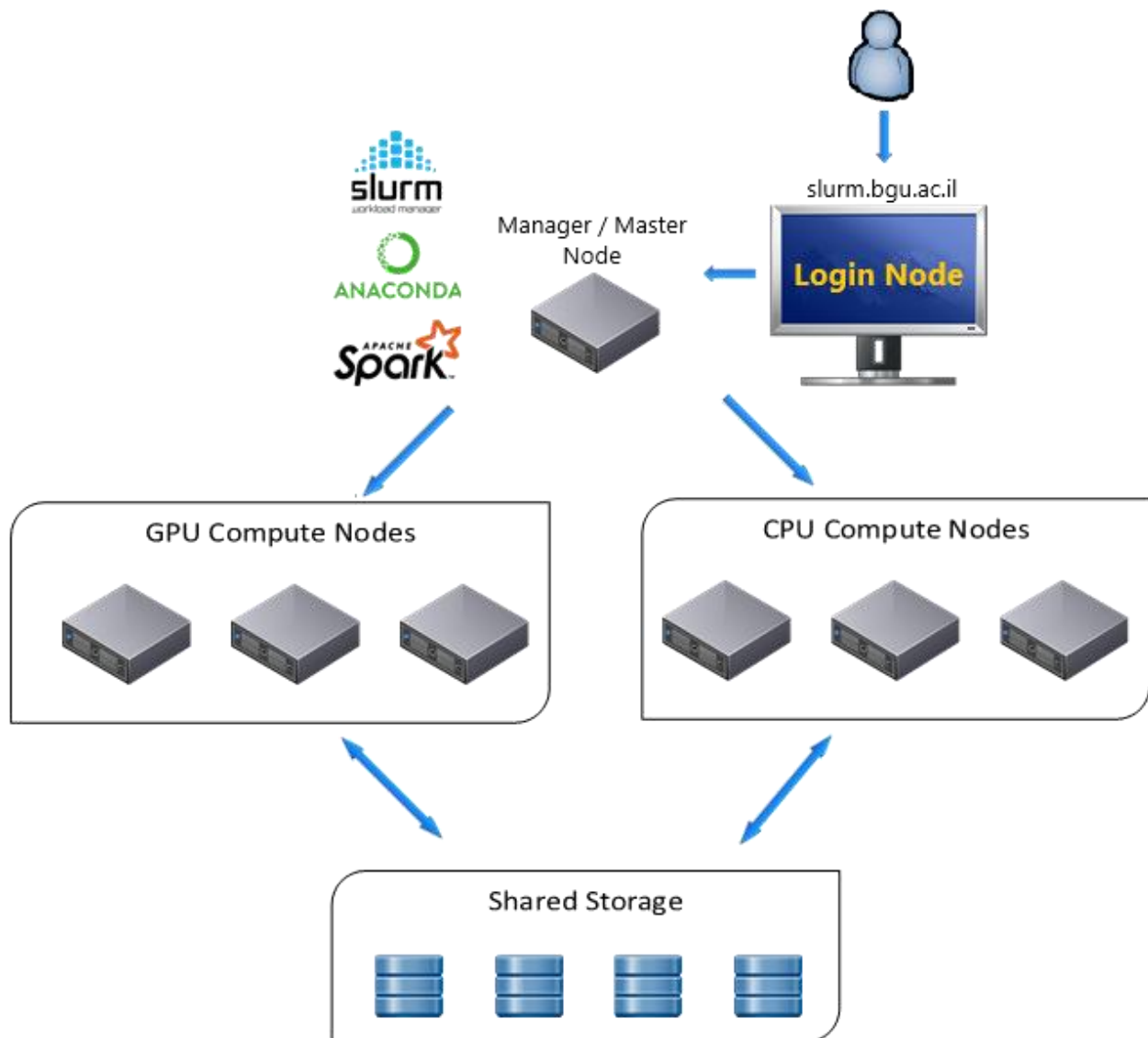
FAQ..... 46

 Usage..... 46

 Errors..... 49

Abstract

This document is located [here](#) and is being updated from time to time. Please make sure you have the most recent version.



BGU ISE, DT and CS departments have a new (2024) Slurm cluster (hpc) that handles both GPU tasks and CPU tasks. Slurm is job scheduler and resource manager used in most of the greatest super computers. The cluster consists of a manager node (also called master node) and several compute nodes (view above illustration).

The manager node is a shared resource used for launching, monitoring, and controlling jobs and should **NEVER** be used for computational purposes.

The compute nodes are powerful Linux servers, some of which are installed with GPUs.

The user connects, by SSH, to the manager node and submits jobs that are executed by a compute node.

A job is allocation of compute resources such as RAM memory, cpu cores, gpu, etc. for a limited time. A job may consist of job steps which are tasks within a job.

In the following pages, *Italic* writing is reserved for Slurm CLI commands.

Use

- Make sure you got admission to the cluster by your IT team.
- Ssh to the Login Node: `slurm.bgu.ac.il`
- Use your BGU user name and password to login to the Login Node. The default path is to your home directory on the storage.
- Python users: create your virtual environment (Conda [Create Environment](#)) on the manager node.
- If you copy files to your home directory, don't forget about file permissions. E.g. for files that need execution permissions, do: `chmod +x <path to file>`
- Remember that the cluster is a shared resource. Currently, users are trusted to act with responsibility in regards to the cluster usage – i.e. **release unused allocated resources (with *scancel*), not allocate more than needed resources, erase unused files and datasets, etc.** Please release the resources even if you are taking a few hours break from interactively using them.
- Anaconda3 is already installed on the cluster. **Do not install it!**
- Should you need tensorflow-gpu package, please do not use *pip install* to install. Rather use: *conda install -c anaconda tensorflow-gpu*
- Please read thoroughly, the following page or two.
- If you are clueless about Linux, Conda and the rest of the environment then use the [Step by Step Guide for First Use](#) page or you may find a video clip guide series in Moodle.
- You may access the 'HPC קלסטר' course in Moodle to find video tutorials (<https://moodle.bgu.ac.il/moodle/course/view.php?id=60163>). Registration password is 'cluster20252'

Submitting a Job

In order to submit a non interactive job, type:

sbatch <your batch file name>

- Conda users: Make sure you submit the job while virtual environment deactivated in the CLI ('conda deactivate')!

To launch interactive jobs, read the chapter [Interactive vs Non-Interactive Use](#).

Batch File

The batch file should look like the following:

There is an example sbatch file located on cluster here: `/storage/example.sbatch`

```
#!/bin/bash

### sbatch config parameters must start with #SBATCH and must precede any other command. to ignore just add another # - like ##SBATCH

#SBATCH --partition main          ### partition name where to run a job. Use 'main' unless qos is required. qos partitions 'rtx3090' 'rtx2080' 'gtx1080'

#SBATCH --time 0-10:30:00        ### limit the time of job running. Make sure it is not greater than the partition time limit (7 days)!! Format: D-H:MM:SS

#SBATCH --job-name my_job        ### name of the job. replace my_job with your desired job name

#SBATCH --output my_job-id-%J.out  ### output log for running job - %J is the job number variable

#SBATCH --mail-user=my_username@post.bgu.ac.il  ### user's email for sending job status notifications

#SBATCH --mail-type=BEGIN,END,FAIL  ### conditions for sending the email. ALL,BEGIN,END,FAIL, REQUEUE, NONE

#SBATCH --gpus=0 ### number of GPUs. Choosing type e.g.: #SBATCH --gpus=gtx_1080:1, or rtx_2080, or rtx_3090. Allocating more than 1 requires the IT team's permission

##SBATCH --tasks=1              # 1 process – more than 1 - for MPI or for processing of few programs concurrently in a job (with srun). Use just 1 otherwise

### Print some data to output file ###

echo "SLURM_JOBID"=$SLURM_JOBID

echo "SLURM_JOB_NODELIST"=$SLURM_JOB_NODELIST

### Start your code below ###

module load anaconda          ### load anaconda module

source activate my_env        ### activate a conda environment, replace my_env with your conda environment

python mycode.py my_arg
```

Should you need the IT team's support, in your request remember to state your username, the job id and include the sbatch file and the output file names and path.

Allocating Resources

Since the resources are expensive and in high demand, you should make use of just **1 GPU per job**.

You should use the **minimum possible RAM**. If your code makes use of 30G then by all means, do NOT ask for 50G! (to get an idea of how much RAM was in use, when the job completes, use: `sacct -j <jobid> --format=JobName,MaxRSS`).

You can only load about 11G to most of the GPUs and 24G to the advanced ones. If your code makes use of 60G then revise it. **Do not allocate more than 60G!**

4-6 CPUs are sufficient to serve the GPU. You should **NOT set that value when allocating a GPU** – this is done by the system.

If your job does not require a GPU then submit it to the CPU cluster.

Running Thousands of Jobs

The Scheduler Overhead Problem

The HPC scheduler is designed to manage complex resource allocations, not to handle a rapid-fire succession of thousands of tiny requests.

- **Latency:** Every time you submit a job, the scheduler must evaluate the priority, check fair-share balances, and communicate with compute nodes. If you submit 5,000 jobs, the scheduler might spend more time "thinking" than your code spends "running."
- **Queue Pending:** Thousands of small jobs can clog the queue, making it difficult for the scheduler to backfill larger jobs and potentially annoying system administrators.

Comparing the Two Approaches

Feature	Thousands of Tiny Jobs	Big Batches (Arrays/Pilots)
Scheduler Load	High (can slow down the cluster)	Low (one entry in the table)
Wait Time	High (each job re-queues)	Low (once the batch starts, it flows)
Data Handling	Risky (I/O metadata bottlenecks)	Managed (can aggregate logs)
Reliability	Hard to track 5,000 logs	Easier to monitor one array

The I/O "Silent Killer"

When running thousands of short jobs, the biggest bottleneck is often the **File System**.

- If 2,000 jobs all try to write a small .log file or open a .csv at the same microsecond, the metadata server on the parallel file system (like Lustre or GPFS) may hang.

- **Best Practice:** Have your jobs write to local /tmp (SSD) on the compute node and move the final results to the shared storage at the end of the batch.

Interactive vs Non-Interactive Use

There are two ways to work with the cluster. Non-interactive way like fire-and-forget – the user specifies the code to be executed and the code shall be executed on a compute node without the user interfering the run. The output to terminal will be directed to a file.

Working interactively with the cluster requires the ssh session to be opened constantly. Once the session is closed it won't be possible to be renewed. This mode is used when working with Jupyter notebooks or IDEs such as PyCharm (refer to relevant chapters). It is possible to directly SSH to the compute node, or even better, in order to connect to a specific job and have access to its environment variables, use: `srun --jobid=<your-jobid> --pty bash`

To launch an interactive job with default values, type: `sinteractive`

To learn about sinteractive script configuration, type: `sinteractive --help`

For example, to launch a 5 hours job allocating a GPU: `sinteractive --time 0-5:00:00 --gpu 1`

Courses users only, use: `sinteractive --qos course --part course --gpu 1`

Do not forget to cancel the job when you are done: `scancel <job_id>`

Please note: there is an issue running multiple simultaneous interactive jobs on the same compute node. Refer to [simultaneous interactive jobs get the same compute node](#) topic in the FAQ.

Information about the compute nodes

sinfo shows cluster information.

sinfo -Nel

NODELIST – name of the node

S:C:T - sockets:cores:threads

List of My Currently Running Jobs

squeue --me

Cancel Jobs

scancel <job id>

scancel --name <job name>

Cancel All Pending jobs for a Specific User

scancel -t PENDING -u <user name>

Running Job Information

Use *sstat*. Information about consumed memory:

sstat -j <job_id> --format=MaxRSS,MaxVMSize

scontrol show job <job_id>

Complete Job Information

sacct -j <jobid>

sacct -j <jobid> --

format=JobName,MaxRSS,AllocTRES,State,Elapsed,Start,ExitCode,DerivedExitcode,Comment

MaxRSS is the maximum memory the job needed.

Resources Usage

sres

Use it to help you make up your mind about which resources to use, when cluster is used at nearly full capacity.

Advanced Topics

You may [Run Jupyter Lab in Udocker](#) . OR

Jupyter Lab

Installation

In your conda environment:

```
conda install jupyterlab
```

Make the Conda Environment Available in Notebook's Interface

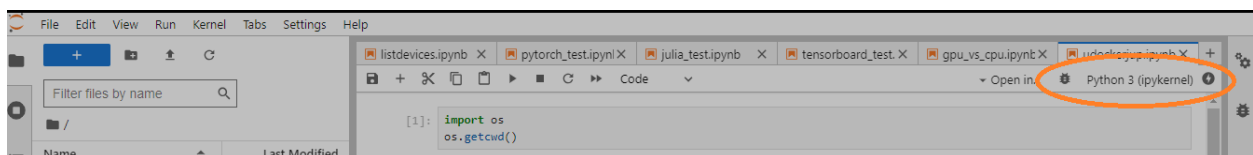
Conda activate your Jupyter installed environment and type:

```
python -m ipykernel install --user --name <conda environment> --display-name "<env name to show in web browser>"
```

e.g.:

```
python -m ipykernel install --user --name my_env --display-name "my best env"
```

Don't forget to choose the right kernel while in the notebook.



Launch Jupyter Lab

Launch jupyter with default values by typing: `sjupyter`

To learn about sjupyter script configuration, type: `sjupyter --help`

For example: `sjupyter --time 0-5:00:00 --gpu 1`

Courses users, use: `sjupyter --qos course --part course --gpu 1`

Wait for resources to be allocated and Jupyter server to be running. Then the script should print a url. Copy the whole url and paste it in your web browser's address bar or just <ctrl> + click the url. Ignore safety warning and allow to proceed (Chrome: press 'advanced' and the 'Proceed to...')

Release Job Resources from Within Jupyter After Code Has Finished Running

Add the following 3 lines at the end of your code to make the code release the job resources when done:

```
import os

job_cancel_str="scancel " + os.environ['SLURM_JOBID']

os.system(job_cancel_str)
```

Tensorboard

No Jupyter

1. Run program and generate logs in my_log_dir (or any other directory)
2. Wait for program run to end
3. ssh to the compute node
4. conda activate my_environment
5. tensorboard --bind_all --logdir=my_log_dir
6. Wait for output. Copy paste link to web browser

In Jupyter

1. In one of the first cells, load the tensorboard extension: `%load_ext tensorboard`
2. After cells generated log files, write in a cell: `!tensorboard --bind_all --logdir=my_log_dir`
3. Wait for output. Copy paste link to web browser

Working with Notebooks

Working with notebooks is interactive. If you closed your browser tab while a notebook's cell is running, it keeps running on the cluster, but you will lose the output. On the other hand, it is not always possible to leave your browser open. The simple solutions to that are either to write variable values and results into a file or to run the code as a python script instead of using a notebook.

In Ipython 6.0 and higher you can use `%capture` cell magic to save all output to file. Use the following line as the very first line of the cell: `%%capture cap_out`

Then, in order to save to variable, on the cell's last line: `var = cap_out.stdout`

If you rather print to file then: `with open('cap_output.txt', 'w') as f:`

```
f.write(cap_out.stdout)
```

When you come back to the notebook you can print the content of `var` or `cap_out.show()` in another cell.

Usefull SBATCH variables

constraint

This variable allows the user to select node feature such as the type of CPU node or type of GPU card

Example for allocating a 128 core CPU node type (it does not mean that 128 cores will be allocated to the job):

```
#SBATCH --constraint=cpu128
```

Available values: cpu, gpu, cpu128, cpu256, gtx_1080, rtx_2080, rtx_3090, rtx_4090, rtx_6000, titan_rtx, tesla_p100

exclude

Exclude node names that you don't want to be allocated. Useful for interactive jobs you don't want to run on the same server.

Example:

```
#SBATCH --exclude=dt-1080-01,ise-1080-02
```

nodelist

List a node name to be allocated for the job. Beware, if you list 2 nodes Slurm will try to allocate both nodes together to the job.

Example:

```
#SBATCH --nodelist=dt-1080-01
```

High Priority Jobs (Golden Tickets)

Some users have the right to prioritize their jobs when the required resources for their job are not available. When you give your job high priority, while submitting it, it might preempt another running job or several running jobs. Also, the prioritized resources are limited and shared among the group users. If a user asks to prioritize a job and the prioritization resources rights are exhausted by the group users, then the job will be pending even though there may be available cluster resources.

The use of high priority jobs is disabled in 'main' partition. User should use another partition in accordance with group's rights for certain resources. For example, if the group's rights include nothing more than 4 2080 type GPUs, then a user in the group can only use the rtx2080 partition for high priority jobs.

Usage:

```
sbatch --partition=<partition name> --qos=<high priority group name> <batch file name>
```

'high priority group name' is usually your instructor's user name.

'partition name' is the name of the partition that goes along with the 'high priority group name'. It may be 'gtx1080', 'rtx2080', 'rtx3090' or 'rtx6000', according to the group's rights.

Example:

```
sbatch --partition=gtx1080 --qos=our_qos my_awesome.sbatch
```

When no QoS is needed, do not use partitions other than 'main'. When using QoS, do not use partition 'main'.

Prioritize Your Own Jobs

Using the 'nice' parameter one may prioritize their own jobs. The 'nice' parameter sets a job's priority lower, so other jobs, of that user, can be prioritized over it. The higher the value is set, the lower the priority it gets.

Default value is 0.

```
scontrol update JobId=<my-job-id> Nice=500
```


Allocate Extra RAM/CPU's

If you are sure that your job requires more than the default 24G RAM per gpu:

In your sbatch file:

To override default ram:

```
#SBATCH --mem=48G
```

If you believe that your job requires more than 58G please contact the IT team.

If you are NOT allocating a GPU and are sure that your job requires more than the default allocation number of cpus:

To override default cpu number

```
#SBATCH --cpus-per-task=6
```

Working with the Compute Node's SSD Drive

You may want to use the compute node local drive for fast access to data. /scratch directory on the compute node is intended for that.

Add to the sbatch script file:

```
#SBATCH --tmp=100G      ### Asks to allocate enough space on /scratch
```

Then in users code section:

```
export SLURM_SCRATCH_DIR=/scratch/${SLURM_JOB_USER}/${SLURM_JOB_ID}
cp /storage/*.img $SLURM_SCRATCH_DIR      ### example of copying .img files TO
node's local storage
mkdir $SLURM_SCRATCH_DIR/testtttt
...
some user code where you write and read to $SLURM_SCRATCH_DIR/testtttt
...
cp -r $SLURM_SCRATCH_DIR $SLURM_SUBMIT_DIR  ### copy back final results to user home
or other accessible location
```

When job has finished, canceled or failed, ALL data in \$SLURM_SCRATCH_DIR is erased! This temp folder lives with running jobs only!

Sending Arguments to sbatch File

Launch job with command line arguments:

```
sbatch --export=ALL,var1='1',var2='hello' my_sbatch_file.sbatch
```

In the sbatch file, use with '\$' e.g.:

```
echo $var2
```

Job Arrays

Job array feature allows you to run identical version of your script with different environment variables. This is useful for parameter tuning or averaging results over seeds.

To use job array, add the following line to your Slurm batch file:

```
#SBATCH --array=1-10  ### run parallel 10 times
```

Adding this will run your script 10 times in parallel, actually creating 10 jobs, where each job gets the requested resources, e.g., if you requested 6 CPUs then each job shall get 6 CPUs. The environment variable SLURM_ARRAY_TASK_ID for each run will have different values (from 1 to 10 in this case). You can then set different parameter setting for each parallel run based on this environment variable.

Remember to change #SBATCH --output=your_output.out to #SBATCH --output=file_name_%A_%a.out, so the output of each parallel run be directed to a different file. %a will be replaced by the corresponding SLURM_ARRAY_TASK_ID for each run. %A will be replaced by the master job id.

To get the above SLURM_ARRAY_TASK_ID variable in python:

```
import os

jobid = os.getenv('SLURM_ARRAY_TASK_ID')
```

In R:

```
task_id <- Sys.getenv("SLURM_ARRAY_TASK_ID")
```

OR

You can send it as argument to the python program like so:

```
python my_code.py $SLURM_ARRAY_TASK_ID
```

Send Name of an Input File to Each Task

For example, if the input files end in .txt

```
file=$(ls *.txt | sed -n ${SLURM_ARRAY_TASK_ID}p)
```

```
myscript -in $file
```

Read a Line from an Input File for Each Task

```
SAMPLE_LIST=($(cat input.list))
```

```
SAMPLE=${SAMPLE_LIST[${SLURM_ARRAY_TASK_ID}]}
```

Email Notifications

If you would like to receive an email for each task in the array, rather than just for the whole job:

```
#SBATCH --mail-type=BEGIN,END,FAIL,ARRAY_TASKS
```

Limiting the Number of Simultaneously Running Tasks from the Job Array

For example, to limit the number of simultaneously running tasks from a 16 jobs job array to 4:

```
#SBATCH --array=0-15%4
```

Job Dependencies

Job dependencies are used to defer the start of a job based on other job's condition.

sbatch --dependency=after:<other_job_id> <sbatch_script> ### start job after other_job started

sbatch --dependency=afterok:<other_job_id> <sbatch_script> ### start job after other_job ends with ok status. E.g. sbatch --dependency=afterok:77:79 my_sbbatch_script.sh -> start after both job 77 and 79 have finished

sbatch --dependency=singleton ### This job can begin execution after the termination of any previously launched jobs, sharing the same job name and user

CUDA Version Selection

CUDA drivers are installed on all compute nodes.

To load a specific version, e.g. 9.0, use the following line in your sbatch file:

```
module load cuda/9.0
```

available versions:

```
cuda/7.0  cuda/7.5  cuda/8.0  cuda/9.0  cuda/9.1  cuda/9.2  cuda/10.0  cuda/10.1  cuda/10.2  cuda/11.0  
cuda/11.1  cuda/11.2  cuda/11.3  cuda/11.4  cuda/11.5  cuda/11.6  cuda/11.7  cuda/11.8  cuda/12.0  
cuda/12.1  cuda/12.2  cuda/12.3  cuda/12.4  cuda/12.5  cuda/12.6  cuda/12.8  cuda/12.9  cuda/13.0  
cuda/13.1
```

IDEs

pyCharm

Make sure you have pyCharm Professional installed (free for students/academy people).

Create an interactive session:

Ssh to Slurm.

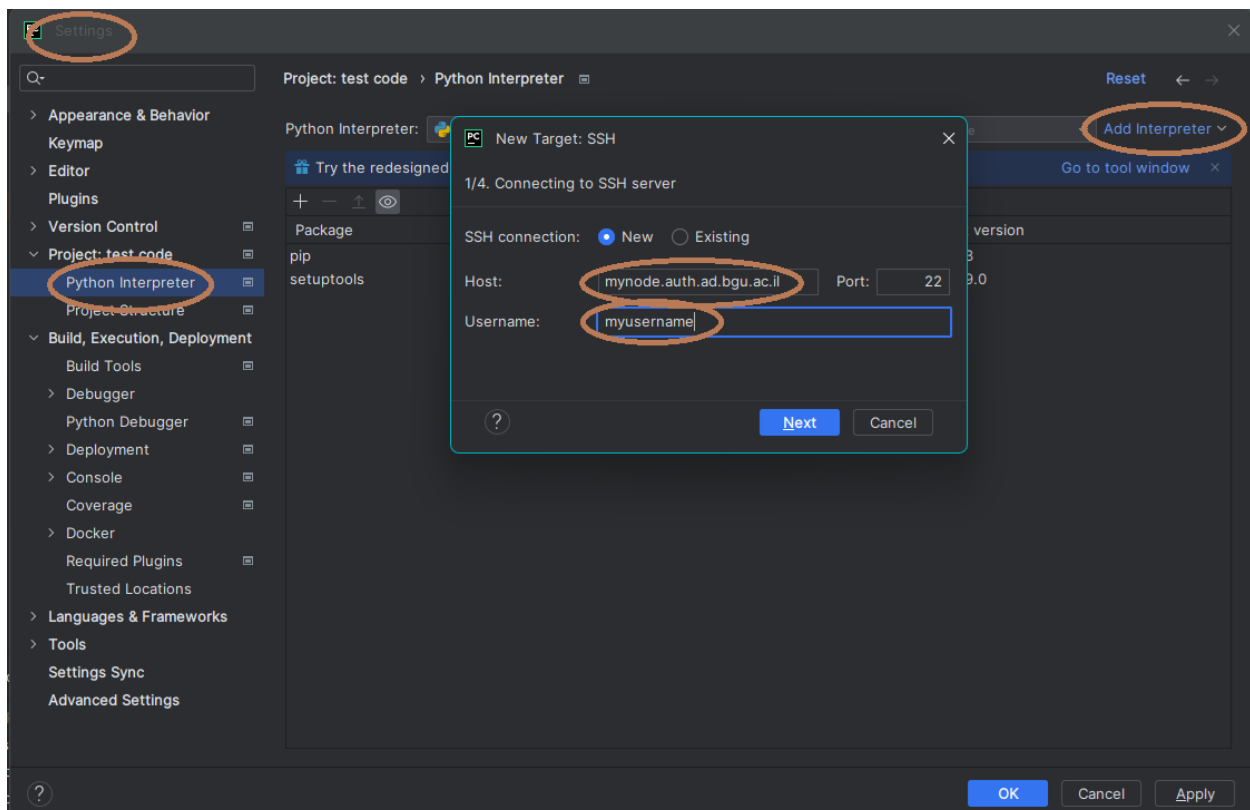
Launch a GPU (or alternatively a CPU job, with no GPU) interactive job by typing: `sinteractive --gpu 1`

The output lists the node's hostname and the job id.

Open pyCharm.

Go to settings->Project->Project Interpreter

On the upper right hand corner next to Project Interpreter: press 'add interpreter' or press the settings icon, choose 'add'.



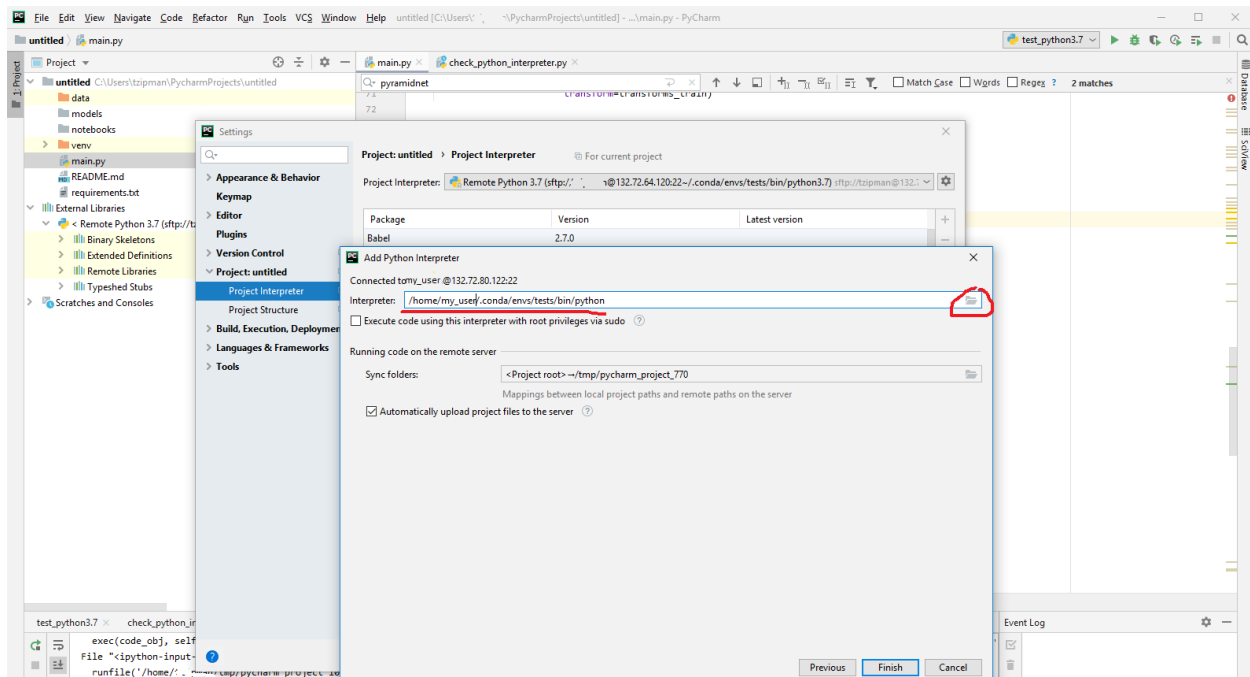
Depending on the version, choose 'On SSH' and then choose 'New' or on the left hand side choose SSH Interpreter. Under 'New server configuration' fill in the **compute node's hostname** (do not fill in the **manager node's hostname!!!**) and your BGU user name. Click Next.

You might get a message about the authenticity of the remote host, asking if you want to continue connecting. Click 'yes'.

Enter your BGU password. Click Next.

Depending on the version, if possible choose 'System Interpreter'.

In the 'Interpreter:' line, enter the path to the desired interpreter. You can find your environments interpreters under `/home/<your_user>/<your_environment>/bin/python`.



Click Finish.

Give pyCharm some time to upload files to the cluster (upload status is shown on the status bar).

- If you don't want to upload your files to the compute node each time you connect to a new compute node, you can map the sync folder to your cluster home directory or subdirectory. You may also opt not to automatically sync files. These are done in the bottom part of the 'Add python interpreter' box that is illustrated above.

Show Remote File Tree Window

Press ALT+F1

Select 'Remote Host'

You may need to go to settings->Build,Execution,Deployment->Deployment and choose the right SSH configuration.

Make phCharm Continue Running Script When Session is Disconnected

The `offline_training.py` script in the frame below, launches another script with arguments:

`train.py --size 192`

The output be redirected into `result.txt` file.

`offline_training.py`

```
import os
import sys

os.system("nohup bash -c '" +
          sys.executable + " train.py --size 192 >result.txt" +
          "' &")
```

The above would run command “`train.py --size 192`”

The `result.txt` file may be found on the compute node. Once you run `offline_training.py` In Pycharm, on the ‘Python Console’ pane, the next line should show up:

```
runfile('/tmp/pycharm_project_<some_number>/<your_offline_running_file>.py',
wdir='/tmp/pycharm_project_<some_number>')
```

The path is the path to the folder in the compute node where your local files are synced. Ssh to the compute node and you may find `result.txt` at that path.

Remember that once the job ends, that folder is erased!

Visual Studio Code

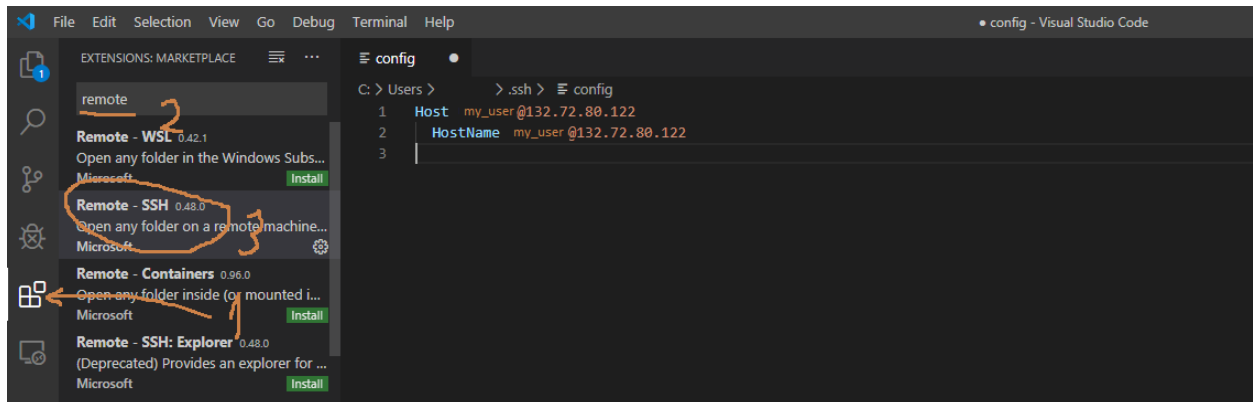
Create an interactive session:

Ssh to Slurm.

Launch a GPU (or alternatively a CPU job, with no GPU) interactive job by typing: `sinteractive --gpu 1`

The output lists the node's hostname and the job id.

Assuming you have VS Code installed and a supported OpenSSH client installed, install the 'Remote – SSH' pack.

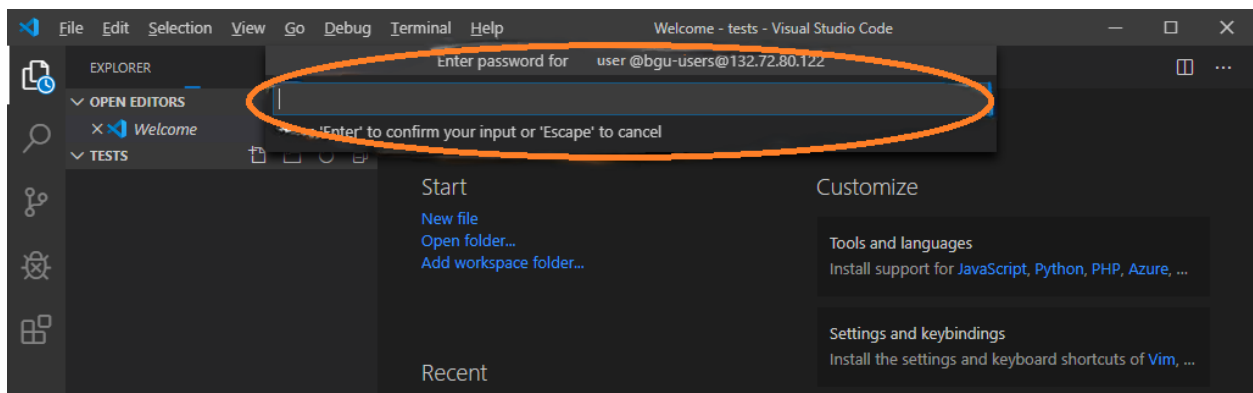


Install the Python package, if needed.

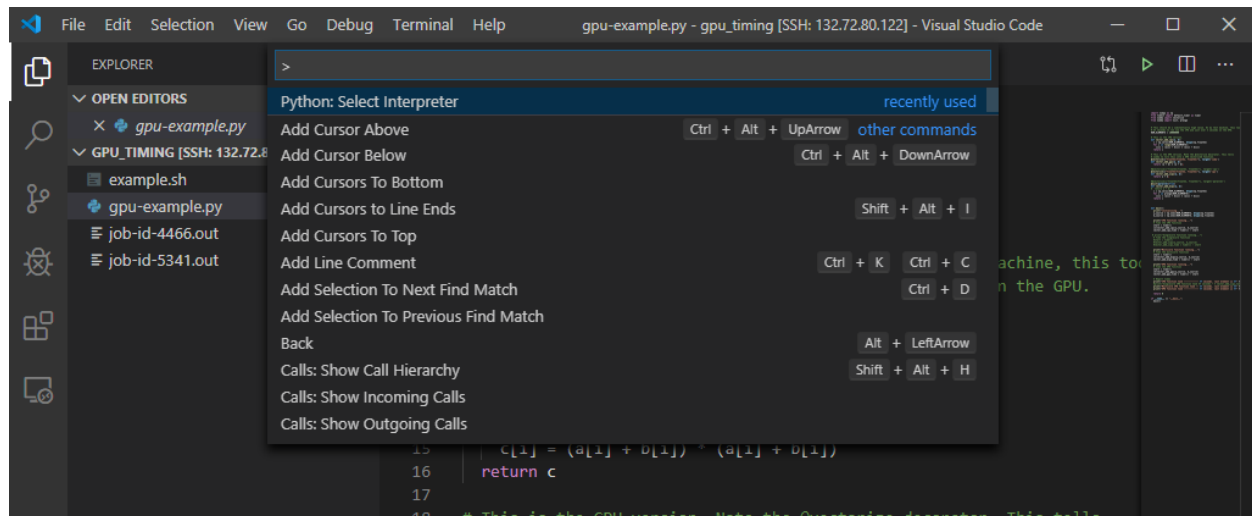
Press the green button (><) on the bottom left corner of the window (under the 'settings' button).

On the middle upper side of the window, choose "Remote – SSH: Connect to host..." and enter `<your_BGU_user>@<compute_node_hostname>`. **Do not fill in the manager node's hostname!!!**

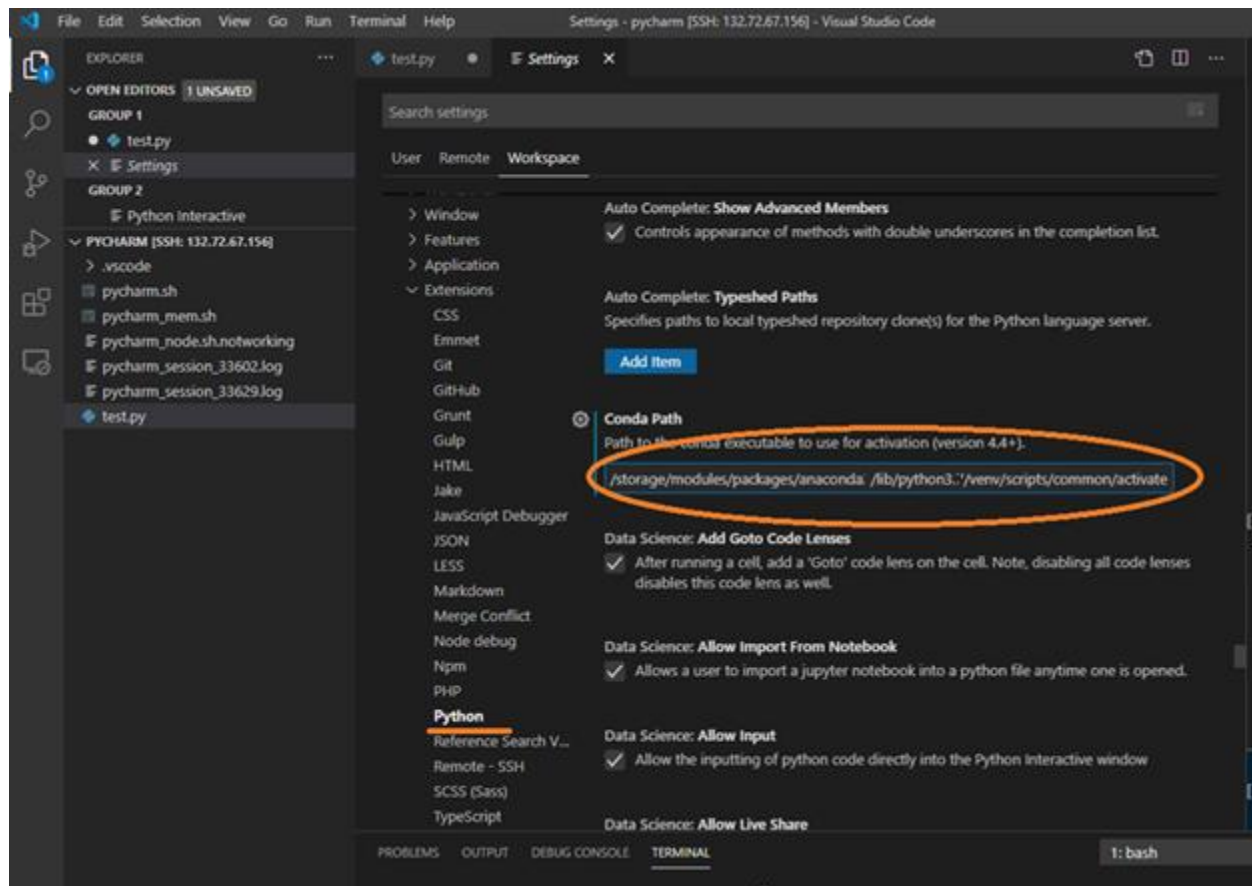
A new window opens. Enter your BGU password, when prompted.



Ctrl+Shift+P for the Command Palette then choose 'Python: Select Interpreter' (start typing – it will show up) and choose the interpreter from your desired environment (~/.conda/envs/<environment>/bin/python).



To enable interactive work with notebook like cells, you may have to: Ctrl+Shift+P for the Command Palette then choose 'Preferences: Open Workspace Settings' (start typing – it will show up) and click 'Python'. Scroll down until you find 'Conda Path' and fill in '/storage/modules/packages/anaconda/lib/python3.11/venv/scripts/common/activate'.



If encountered an errata with finding the actual path of the python script then add the following line to launch.json file:

```
"cwd": "${fileDirname}"
```

Refer to the following paragraph for instructions as to how to place it in the file and where.

[Run/Debug with Arguments](#)

Open the python file to be launched with arguments. Press the Debug symbol on the left vertical ribbon. Click 'create a launch.json file' on the left pane.

Open file launch.json and add another line within 'configurations' like so (example for 4 arguments):

```
"args": ["-arg_name1", "value_1", "-arg_name2", "value_2"]
```

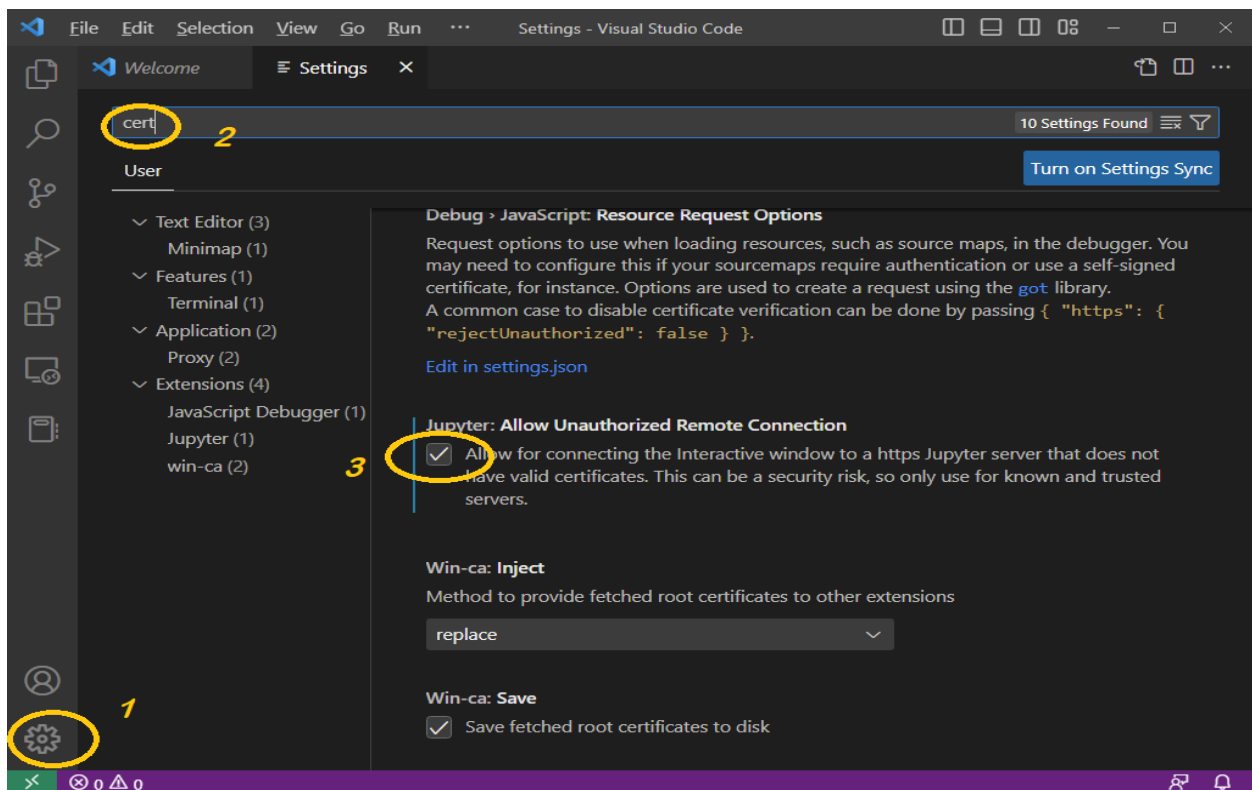
Here is an example:

```
"configurations": [  
  {  
    "name": "Python: Current File",  
    "type": "python",  
    "request": "launch",  
    "program": "${file}",  
    "console": "integratedTerminal",  
    "cwd": "${fileDirname}",  
    "args": ["cuda", "100", "exit"]  
  }  
]
```

Run Jupyter Notebook

To avoid SSL certificate error, do the following:

1. Press the cogwheel then select 'Settings', start writing 'cert', scroll all the way down and tick 'Jupyter: Allow Unauthorized Remote Connection'.
2. Edit cluster file ~/.vscode/settings.json -> add the line: "http.systemCertificates": true
3. Add the following line at the end of cluster file ~/.bashrc:
export NODE_TLS_REJECT_UNAUTHORIZED='0'



Docker

Running Docker containers on the cluster may be done using either the preferred Apptainer or UDOCKER.

Apptainer

- Create an interactive job using the following script: `/storage/interactive.sh`
- Ssh to the compute node that was allocated to you
- Download your desired container and create a .sif file:
`apptainer build --force my_container.sif docker://my_container_page/my_container`
- Run the container (the last argument is a command. In this example, it starts a bash shell):
`apptainer exec my_container.sif /bin/bash`

OR

```
apptainer exec --nv --bind cluster_dir:container_dir my_container.sif /bin/bash
```

where `--nv` - gives the container access to the GPU (if allocated to the job)

`--bind` – bind a cluster directory to a container directory – useful for write permissions

Building Images on Your Local Machine

- Build the image with Docker on your system
- Save the image to a .tar file on your system: `"docker save -o my-image.tar my-image:latest"`
- Copy the .tar file to the cluster
- Run a job on the cluster that builds the .sif image and uses it with Apptainer: `"apptainer build my-image.sif docker-archive://my-image.tar"`

UDOCKER

UDOCKER shall be installed in a Conda environment.

UDOCKER is not a full Docker replacement and usage is currently **limited to pulling and running containers**. The actual containers should be built using Docker and dockerfiles.

UDOCKER is not a full Docker replacement and usage is currently **limited to pulling and running containers**. The actual containers should be built using Docker and dockerfiles.

Installation

- Installation for Python3 udocker. For Python2: remove the “python=3.8” string in the next line.

```
conda create -n udocker_env python=3.8
```

```
conda activate udocker_env
```

```
conda install configparser
```

```
pip install udocker
```

Test

For example: test a tensorflow-gpu container. Copy paste the following at the end of an sbatch file:

Once udocker environment is activated you can use:

```
module load anaconda
source activate udocker_env
udocker pull tensorflow/tensorflow:2.8.0rc0-gpu-jupyter # pull image
udocker create --name=tf_gpu_jup28 tensorflow/tensorflow:2.8.0rc0-gpu-jupyter # create and name container
udocker setup --nvidia tf_gpu_jup28 # setup GPU support
udocker run tf_gpu_jup28 nvidia-smi # run container with command 'nvidia-smi'
# mount your code directory to container's /home directory and run your python code
udocker run -v /home/my_user/my_code_dir:/home tf_gpu_jup28 python3 /home/my_code.py
```

udocker --help – info about commands and way of use

udocker run --help – help for the ‘run’ command. This can be done also with other commands

udocker ps – list your containers

udocker images – list your images

udocker rm <container name/id> - remove container

udocker rmi <image id> - remove image

There is no need to pull the image every time.

There is no need to create the container if it already exists. No need to setup Nvidia, for that container, either, if that was already done.

Run Jupyter Lab in Udocker

- After [Error! Reference source not found.](#) of a udocker conda environment as explained, copy file /storage/udocker_jup.sbatch to your directory
- Modify the environment name and the docker image as desired.

- Run: `sbatch udocker_jup.sbatch`
- Open the output file and grab the port number from the first lines. Grab the token from the last lines of the file (it may take some time before the image is downloaded and the token displayed. For the example image it took 15 minutes)
- Replace the port number of the token with the port number you grabbed.
- Replace the hostname with its IP address (you can ping to it to have its IP address displayed)
- Paste the modified token in your favorite web browser's address bar and hit <Enter>

Matlab

Run Matlab GUI straight from terminal (no need for sbatch):

```
module load matlab
```

```
srun --x11 --nodes=1 --mem=24G --cpus-per-task=4 --gpus=1 --partition=main matlab -nosftwareopengl  
-desktop -sd ~
```

For Matlab 2021A:

```
module load matlab/R2021A
```

```
srun --x11 --nodes=1 --mem=24G --cpus-per-task=4 --gpus=1 --partition=main --time=01:00:00 matlab -  
desktop -sd ~
```

Make sure your ssh terminal supports x11 forwarding!

Send Matlab script to execute as batch by headless Matlab:

```
srun --nodes=1 --mem=24G --cpus-per-task=4 --gpus=1 --partition=main matlab -nosplash -nodisplay -  
nodesktop -sd ~ -batch "my_matlab_script"
```

Headless Matlab may be run by sbatch as well.

- Cluster params:
 - --nodes – number of allocated cluster nodes (must be 1)
 - --mem=24G – memory allocation
 - --cpus-per-task – number of CPUs
 - --gpus – number of GPUs
 - --partition – partition name
- Matlab params:
 - -desktop – run matlab in GUI mode
 - -sd – matlab working directory (user's home directory)
- Check allocation in Matlab console

```
gpuDeviceCount
```

```
feature('numcores')
```

julia

First time installation:

```
julia -e 'using Pkg;Pkg.add("IJulia")'
```

The Julia kernel and Julia console will be available in Jupyter Notebook.

R

Command Line

- Create an R conda environment. E.g.: `conda create -n r_env r-essentials r-base`
- Launch an interactive job: `sinteractive` (use `sinteractive --help` to learn about job options)
- Wait for the resources to be allocated.
- Copy the compute node's hostname from the script output.
- SSH to the compute node.
- Type: `conda activate r_env`
- Type: `R`

R in Jupyter

An example for creating an R conda environment:

```
conda create -n r_jupyter python=3.9 jupyterlab r-essentials r-base
```

```
conda activate r_jupyter
```

```
conda install -c conda-forge r-irkernel
```

```
R -e "IRkernel::installspec()"
```

In Jupyter web interface, change kernel and select “R”

R in MS Visual Studio Code

Install the R extension for Visual Studio Code. A new ‘R’ button will show up on the left menu.

Open an ipynb notebook.

If you wish to run .r files then follow these steps:

- Press <Ctrl> + ‘,’ for settings
- Scroll down and find ‘R’ on the left menu
- Scroll down the ‘settings’ window and find ‘R>Rpath: Linux’. Type the location of the R executable: `/home/<your_user_name>/conda/envs/<your R environment>/bin/R`
- Repeat the previous step for ‘R>Rterm: Linux’

RStudio

You can run an RStudio Apptainer and connect to it using a web browser.

- Go to your working directory.
- Copy the scripts: `cp /storage/scripts/apptainer/rstudio/`.
- Launch job: `sbatch rstudio.sbatch`
- Extract the IP address and allocated port from the output file and paste in your web browser's address bar: `132.72.X.Y:port-number`

C#

Install In Conda Environment

Activate your conda environment.

```
conda install -c conda-forge dotnet-sdk
```

Use

- Use `./interactive.sh` script to allocate a compute node.
- ssh to the compute node
- Activate environment: *conda activate <your dotnet environment name>*
- Create a new dotnet project: *dotnet new console -o myApp*
- *cd myApp*
- Run: *dotnet run*

Adding Packages to .Net

```
dotnet add package <package name>
```

Adding Project to Solution

```
dotnet sln my_solution.sln add some_project.csproj
```

Fiji – Image Analysis Tool

You can read about Fiji here: <https://fiji.sc/>

Run Fiji on the cluster

```
srun --x11 --gpus=1 --partition=gtx1080 /storage/apps/Fiji/ImageJ-linux64
```

Make sure you use ssh terminal that supports X11 forwarding!

Appendix

Step by Step Guide for First Use of Python and Conda

1. Make sure you are connected through VPN or from within BGU campus.
2. Download a SSH terminal (<https://mobaxterm.mobatek.net/download.html>).
3. Open the SSH terminal and start a SSH session (port 22). The remote host is `slurm.bgu.ac.il`. The username is your BGU username and the password is your BGU password.
4. Once logged into the cluster's manager node, create your Conda environment. E.g.:

```
conda create -n my_env python=3.10
```

5. **conda activate my_env**
6. **pip install <whatever package you need>** or **conda install...**

- It is advisable to use pip install for pytorch. E.g.:

```
pip3 install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118
```

- For Tensorflow: `pip install 'tensorflow[and-cuda]'`

7. **conda deactivate**
8. Copy the sbatch file (job launching file) by typing (do not forget the dot at the end!):

```
cp /storage/example.sbatch .
```

9. Edit the file using nano editor: **nano example.sbatch**
10. You may change the job name by replacing `my_job` with your own string.
11. Go to the last lines of the file. 'source activate my_env': if needed, replace 'my_env' with your environment name that you have created on paragraph 4.
12. The very last line demonstrates running `my_code.py` python file with 1 line argument
13. Press '<ctrl>+x', then 'y' and '<Enter>' to save and leave the file.
14. Launch a new job: **sbatch example.sbatch**
15. You should, instantly, get the job id.
16. To see the status of your job(s) type **squeue --me**
17. Under 'ST' (state) column if the state is 'PD' then the job is pending. If the the state is 'R' then the job is running and you can look at the output file for initial results (jupyter results will take up to a minute to show): **less job-<job id>.out**

Example for Creating Latest Tensorflow-gpu and Jupyter Lab Environment

In order to get the **latest** Tensorflow version, you need to install it on a compute node (and **not** on the master node!)

- Submit an interactive job: `sinteractive --gpu 1`
- Wait for the job to run. Once you have the compute node's hostname, ssh to it.
- Create new environment named 'tfgpu_jup', with tensorflow-gpu: `conda create -n tfgpu_jup`
- Activate new env: `conda activate tfgpu_jup`
- Install tensorflow with GPU support: `pip install 'tensorflow[and-cuda]'`
- Install Jupyter Lab: `conda install -c conda-forge jupyterlab`
- Make the Conda Environment Available in Notebook's Interface: `python -m ipykernel install --user --name tfgpu_jup --display-name "tfgpu_jup"`
- Deactivate new environment: `conda deactivate`
- Cancel interactive job: `scancel <job id>`
- Submit a new jupyter job request: `sjupyter --gpu 1`
- wait for the resources to be allocated. Copy the whole token (address 132.72.X.Y) and paste in your favorite web browser's address bar.
- Confirm advancing in spite of the security warning.
- Create a new notebook.
- Select tfgpu_jup kernel from the upper right corner of the notebook.

Conda

Viewing a list of your environments

```
conda env list
```

list of all packages installed in a specific environment

```
conda list
```

to see a not activated environment

```
conda list -n <my_env>
```

Activating / deactivating environment

```
source activate <my_env>
```

or (depends on conda version)

```
conda activate <my_env>
```

```
conda deactivate
```

Create Environment

```
conda create -n <my_env>
```

with specific python version

```
conda create -n <my_env> python=3.4
```

with specific package (e.g. scipy)

```
conda create -n <my_env> scipy
```

Or

```
conda create -n <my_env> python
```

```
conda install -n <my_env> scipy
```

with specific package version

```
conda create -n <my_env> scipy=0.15.0
```

with multiple packages

```
conda create -n <my_env> python=3.4 scipy=0.15.0 astroid babel
```

Remove Environment

```
conda env remove --name myenv
```

Update Conda

```
conda update conda
```

Compare Conda Environments

The following python (2) script compares 2 conda environments and can be found in '/storage' directory.

```
python conda_compare.py <environment1> <environment2>
```

Transfer Files

To / From Your PC

You can use WinSCP to transfer files.

Get a Public File

from AWS s3

Use wget:

```
wget --no-check-certificate --no-proxy 'https://<your bucket name>.s3.amazonaws.com/<path and name of file>'
```

from Google Drive

Use wget:

```
wget --load-cookies /tmp/cookies.txt  
https://docs.google.com/uc?export=download&confirm=\$\(wget --quiet --save-cookies /tmp/cookies.txt --keep-session-cookies --no-check-certificate 'https://docs.google.com/uc?export=download&id=<YOUR FILE ID>' -O- | sed -rn 's/. \*confirm=\(\[0-9A-Za-z\_\]+\). \*/\1\n/p'\)&id=<YOUR FILE ID>"\)
```

where <YOUR FILE ID> is the alphanumeric long string that shows when you right click the file in Chrome and view the file's link.

Github

How to Set an SSH Key to Your Account

- If you didn't generate ssh keys in your home directory, generate them now:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

- When you're prompted to "Enter a file in which to save the key", press **Enter** to accept the default file location.
- When prompted for passphrase, type an easy to remember passphrase as you will need it every time you connect to Github.
- To copy the key to your github account, print it: `cat ~/.ssh/id_ed25519.pub`
- Copy the key to your clipboard and paste it in your github account ssh keys page: <https://github.com/settings/keys>. Press the 'create a new ssh key' button and paste the key there.
- To test the connection (you will have to type your passphrase): `ssh -T git@github.com`

FAQ

Usage

- ❖ Can I ssh the cluster when I am away from university?

This can be done by using VPN.

- ❖ I uploaded files to the cluster, while logged in to the manager node, can a compute node find these files?

The files were uploaded to the storage. **All** cluster nodes have access to your files on the storage

- ❖ I need sudo to install library X / tool Y

Install it in your conda environment, using 'conda install' or 'pip install'

- ❖ How to tell which GPU was allocated for the job?

Write '*nvidia-smi -L*' either in the sbatch file or ssh to the compute node and run it there.

- ❖ Even though I installed tensorflow, it does not recognize any gpu.

Make sure to: `pip install 'tensorflow[and-cuda]'`

- ❖ Does a Jupyter notebook keep on running when I close the browser?

Please refer to [Working with Notebooks](#).

- ❖ When running Jupyter lab on my browser, kernel shows as disconnected?

Try using another browser or reset browser to factory defaults. Sometimes browser add ons may prevent the browser from properly communicating with kernel.

- ❖ With Mac OS, when opening Jupyter lab on Chrome or Safari browsers, security settings prevent the notebook from loading.

Use Firefox or Maxthon browsers.

- ❖ Is Git installed on the cluster?

Git is installed on the manager node.

- ❖ Why is my job pending? What's the meaning of REASON?

PartitionTimeLimit – the 'time' variable in your sbatch file is set to time limit greater than the partition's maximum possible time limit (usually 7 days).

Resources – currently, the cluster has insufficient resources to fulfill your job.

Priority – job is queued behind higher priority jobs. You may have exceeded your group QoS priority resources – You can launch a job with no QoS priority or wait for QoS priority resource to be available.

QOSMaxJobsPerUserLimit – you have reached the maximum allowed concurrent jobs for the requested partition.

MaxGRESPerAccount – your requested high priority job exceeds the limit of concurrent gpus allocated to your account. Job is waiting for golden card to be available.

- ❖ In python app I print some run time info but they are buffered and being printed all at once.

To use unbuffered print to output: `python -u my_py_app.py`

u – unbuffered.

Another option is to add the following line to your sbatch scripy:

```
export PYTHONUNBUFFERED=TRUE
```

Please note it has performance toll, so it is not advisable to use it when not debugging.

- ❖ Tensorflow does not recognize GPU

Make sure the tensorflow version supports GPU. Make sure libraries and driver versions match.

VERSION	PYTHON	COMPILER	BUILD TOOLS	CUDNN	CUDA
tensorflow-2.15.0	3.9-3.11	Clang 16.0.0	Bazel 6.1.0	8.9	12.2
tensorflow-2.14.0	3.9-3.11	Clang 16.0.0	Bazel 6.1.0	8.7	11.8
tensorflow-2.13.0	3.8-3.11	Clang 16.0.0	Bazel 5.3.0	8.6	11.8
tensorflow-2.12.0	3.8-3.11	GCC 9.3.1	Bazel 5.3.0	8.6	11.8
tensorflow-2.11.0	3.7-3.10	GCC 9.3.1	Bazel 5.3.0	8.1	11.2
tensorflow-2.10.0	3.7-3.10	GCC 9.3.1	Bazel 5.1.1	8.1	11.2
tensorflow-2.9.0	3.7-3.10	GCC 9.3.1	Bazel 5.0.0	8.1	11.2
tensorflow-2.8.0	3.7-3.10	GCC 7.3.1	Bazel 4.2.1	8.1	11.2
tensorflow-2.7.0	3.7-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.6.0	3.6-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.5.0	3.6-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.4.0	3.6-3.8	GCC 7.3.1	Bazel 3.1.0	8.0	11.0
tensorflow-2.3.0	3.5-3.8	GCC 7.3.1	Bazel 3.1.0	7.6	10.1
tensorflow-2.2.0	3.5-3.8	GCC 7.3.1	Bazel 2.0.0	7.6	10.1
tensorflow-2.1.0	2.7, 3.5-3.7	GCC 7.3.1	Bazel 0.27.1	7.6	10.1
tensorflow-2.0.0	2.7, 3.3-3.7	GCC 7.3.1	Bazel 0.26.1	7.4	10.0
tensorflow_gpu-1.15.0	2.7, 3.3-3.7	GCC 7.3.1	Bazel 0.26.1	7.4	10.0
tensorflow_gpu-1.14.0	2.7, 3.3-3.7	GCC 4.8	Bazel 0.24.1	7.4	10.0

To load cuda driver, write “module load cuda/xx.x” where module anaconda is loaded in your job submission file. xx.x is the cuda version to load. Refer to [CUDA Version Selection](#).

- ❖ My 2 simultaneous interactive jobs get the same compute node. All SSH sessions to that node show the same jobid thus use that job's resources and I want one of the sessions to use the other job's resources.

The SSH session uses the first job's resources.

In order to connect to the second job resources, type: `srunc --jobid=<my-2nd-jobid> --pty bash`

You can also type the above at the VS Code terminal.

The above opens a new shell. To close that shell, type: `exit`

- ❖ My job requires a lot of RAM, what can I do?

Find the reason for consuming so much RAM. For example if you are working with a pictures dataset and the preprocessing of the dataset consumes a lot of RAM, then use pointers to pictures for preprocessing the pictures themselves, if possible, as demonstrated here:

<https://www.kaggle.com/itslek/transfer-learning-keras-flowers-sf-dl-v1>

- ❖ How to profile python memory usage?

<https://www.pluralsight.com/blog/tutorials/how-to-profile-memory-usage-in-python>

- ❖ Using Bert consumes a lot of RAM and either OOM errors occur or a lot of RAM needs to be allocated.

Follow the following links to learn how to reduce memory consumption when working with Bert:

github.com/google-research/bert#out-of-memory-issues

stackoverflow.com/questions/59617755/training-a-bert-based-model-causes-an-outofmemory-error-how-do-i-fix-this

- ❖ I need java version 11, but conda install openjdk, installs a version called 1.8

The following will install java 11 in your home folder:

Create a conda environment and pip install `install-jdk==0.1.0`

In python:

```
import jdk  
  
jdk.install('11')
```

For using this java version, you have to modify environment variables (e.g. version 11.0.22):

```
export JAVA_HOME="/home/<your username>/.jdk/jdk-11.0.22+7"  
  
export PATH=$JAVA_HOME/bin:$PATH
```


Errors

- ❖ “RuntimeError: CUDA out of memory. Tried to allocate 448.00 MiB (GPU 0; 10.73 GiB total capacity; 9.64 GiB already allocated; 124.69 MiB free; 195.47 MiB cached)”

Or “Resource exhausted: OOM when allocating tensor with shape...”

The problem arises when trying to allocate more GPU memory than available (e.g. 10.73GiB for Nvidia 2080). Try to reduce the size of the batch you load to the GPU in order to fit the GPU’s available memory.

If you have done the above and still get this error, it may be related to the code grabbing more memory than you realize.

Working with **Tensorflow** – Tensorflow grabs 95% of the memory as default (to avoid performance costly dynamic allocation), so when trying to allocate more memory (sometimes from another process) you will get this error.

To configure **Tensorflow** to allocate growing amount of memory (flexible but performance costly) (**Tensorflow1** apis. For **Tensorflow2** there is a compatibility module explained at the end of the paragraph):

```
config = tf.ConfigProto()
config.gpu_options.allow_growth = True
session = tf.Session(config=config, ...)
```

Or:

```
tf.config.experimental.set_memory_growth(physical_devices[0], True)
```

To configure Tensorflow to allocate another fixed size (e.g. 1/3) of fraction of memory per process:

```
gpu_options = tf.GPUOptions(per_process_gpu_memory_fraction=0.333)
sess = tf.Session(config=tf.ConfigProto(gpu_options=gpu_options))
```

For **Tensorflow2** compatibility module replace *tf.GPUOptions* with *tf.compat.v1.GPUOptions*, replace *tf.ConfigProto* with *tf.compat.v1.ConfigProto* and so on.

Working with **pyTorch** – a) sometimes you leave a reference to a tensor(s) in Cuda. That will make cuda keep the memory for the tensor and if you are iterating then it accumulates (refer to responder ‘samkellerhals’ in <https://github.com/pytorch/pytorch/issues/16417>). b) Use *detach()* if appropriate (to remove the graph associated with the tensor if you don’t use gradient descent). c) You can also use the garbage collector and cache emptying, in your code, as follows:

```
del variables
gc.collect()
torch.cuda.empty_cache()
```

In order to have a readable summary of allocation of memory in the gpu use:

```
torch.cuda.memory_summary(device=None, abbreviated=False)
```

More reading: <https://pytorch.org/docs/stable/notes/faq.html>

- ❖ Got: `RuntimeError: CUDA error: no kernel image is available for execution on the device...`

This error results from CUDA code that was not compiled to target your GPU architecture. The 1080 GPU architecture is a bit outdated – don't use it for that code.

- ❖ Running Jupyter in VS Code I get the following error: Failed to start the Kernel. request to `https://some.ip.and:port/api/sessions?a_number` failed, reason: self signed certificate. View Jupyter [log](#) for further details.

Make sure you use the settings described in chapter [Run Jupyter Notebook](#).

- ❖ I installed wget python package like so: 'conda install wget', and I get 'ModuleNotFoundError: No module named wget' error.

Use: `pip install wget`

- ❖ In my conda environment I installed a package which is a python wrapper of binary code. Running a code that uses it, with Jupyter notebook was successful, but running the very same code from pycharm, failed with the message 'NotImplementedError: "... does not appear to be installed or on the path, so this method is disabled. Please install a more recent version of ... and re-import to use this method.'

This happens because with pycharm, the environment variable 'PATH' remains unchanged, unlike with Jupyter that when choosing conda environment, 'PATH' gets modified. The solution is to modify 'PATH' **prior** to **importing** the wrapper package in your python code, as follows (replace <your_user> and <your_env> with yours):

```
import os
os.environ['PATH'] =
'/home/<your_user>/anaconda3/bin:/storage/modules/packages/anaconda3/bin:/storage/modules/bin:/storage/modules/packages/anaconda3/condabin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/storage/modules/packages/matlab/R2019B/bin:/home/<your_user>/local/bin:/home/<your_user>/bin'
import <python wrapper package>
...
```

- ❖ Using DDP in PyTorch with multiple RTX6000 cards the program hangs. This is not showing on 3090 and 4090 cards.

This is due to server topology that does not support P2P. Add the following to your sbatch file, after the #SBATCH lines:

```
export NCCL_P2P_DISABLE=1
```

- ❖ Using PyTorch I get the following error: `OSError: [Errno 28] No space left on device`

This indicates that tmpfs (folder /dev/shm) was full. This folder is used for temporary RAM files and is configured to be half the size of the node's RAM. You can try either:

limit dataset size,

or disable multiprocessing by setting `num_workers=0`,

or direct the data to the file system temporary folder (you can also set it by: `export TMPDIR=/your/custom/temp/folder` and it is advised to set it to the scratch folder: [Working with the Compute Node's SSD Drive](#)):

```
import torch.multiprocessing as mp
mp.set_sharing_strategy('file_system')
```

- ❖ Using GPU 3090 with tensorflow, I get the following error: `InternalError: CUDA runtime implicit initialization on GPU:0 failed. Status: device kernel image is invalid`

You need tensorflow > 2.2.

- ❖ Using GPU 3090 with pytorch, I get the following error: `NVIDIA GeForce RTX 3090 with CUDA capability sm_86 is not compatible with the current PyTorch installation.`

The current PyTorch install supports CUDA capabilities sm_37 sm_50 sm_60 sm_70.

If you want to use the NVIDIA GeForce RTX 3090 GPU with PyTorch...

Pip upgrade your torch version: `pip3 install torch==1.10.1+cu113 torchvision==0.11.2+cu113 torchaudio==0.10.1+cu113 -f https://download.pytorch.org/whl/cu113/torch_stable.html`

- ❖ VSCode error: Could not establish connection to... The VS Code Server failed to start SSH

This is an SSL certificate error on the client side. Make sure SSH package of VS Code is updated and follow the instructions described in chapter [Run Jupyter Notebook](#).

- ❖ VSCode error: windows remote host key has changed port forwarding is disabled

OR: Could not establish connection to "x.x.x.x": Remote host key has changed, port forwarding is disabled.

OR: visual studio code could not establish connection... the process tried to write to a nonexistent pipe

These errors mean that the remote host key does not match the key saved locally, anymore. The keys mismatch may be a result of reinstalling the remote host. If that is the reason, go to

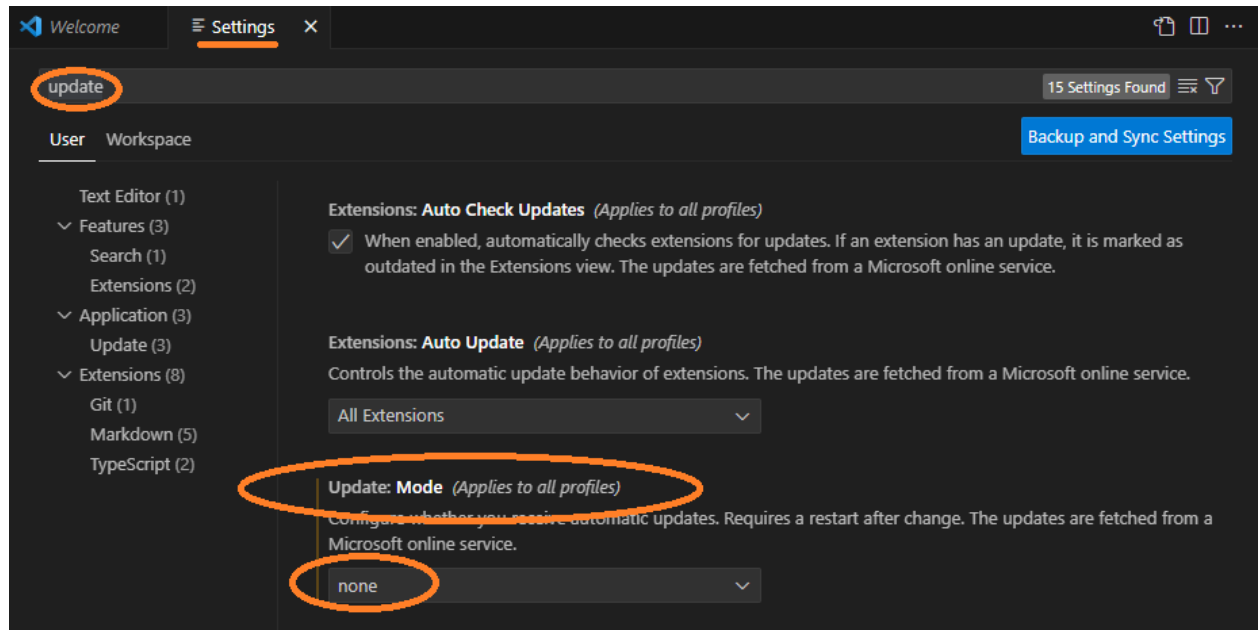
C:\Users\

❖ VSCode glibc errors when connecting to cluster

New versions from 1.86 on require advanced glibc versions which are currently not available for the cluster. Uninstall VSCode and install VSCode 1.85 from here:

update.code.visualstudio.com/1.85.2/win32-x64-user/stable

Then, immediately, disable auto update by: File -> settings, type “update” and change “Update: Mode” value to “none”. See picture below.



❖ Cannot find kernels when running notebooks in VS Code after reinstalling VS Code

Your profile may be corrupted. File->Preferences->Profile->Show Profile Content and inspect the packages for irregularities. You can also create a new profile in that menu.

❖ When I set the job's request like so: `SBATCH --gpu=rtx_6000:2` `SBATCH --nodes=1` I get the following error: `sbatch: error: GPU Parameter Set ! Using GPU Partition. sbatch: error: Batch job submission failed: Requested node configuration is not available`

There is a system erratum. Add: `SBATCH --cpus-per-gpu=8`

❖ `NotImplementedError: Cannot convert a symbolic Tensor... to a numpy array`

Downgrade numpy version to 1.19: `pip install numpy==1.19.5`

- ❖ When I run the Fiji srun command, I get the following message “srun: error: No DISPLAY variable set, cannot setup x11 forwarding.”

This happens when you use ssh terminal that does not support x11 forwarding. Use a terminal that does support it, such as MobaXterm.

- ❖ Output file shows: slurmstepd: error: _is_a_lwp: open() /proc/60830/status failed: No such file or directory

It's a rare Slurm accounting error message that should not affect the job. Just ignore it.

- ❖ I get RuntimeError: CUDA error: no kernel image is available for execution on the device CUDA kernel errors might be asynchronously reported at some other API call,so the stacktrace below might be incorrect. For debugging consider passing CUDA_LAUNCH_BLOCKING=1

This error is a result of incompatible pyTorch version. Update installed version.

- ❖ Using pytorch I get: RuntimeError: CUDA error: device-side assert triggered

Run your tensors on CPU instead of GPU and set CUDA_LAUNCH_BLOCKING=1 to get the exact location and nature of the error. Usually the problem arises as a result of out of range indexing.

- ❖ When connecting with PyCharm to remote server, I get 'FileNotFoundError: [Errno 2] No such file or directory: '/tmp/pycharm_project_xxx/Main.py'

After finishing a job with PyCharm, saved data in PyCharm:

- (1) Tools -> Deployment -> Configuration -> Remove All IP Cache
- (2) Tools -> Deployment -> Configuration -> SSH Configuration -> three dots... -> Remove All IP Cache
- (3) File -> Invalidate Caches and Restart
- (4) Interpreter -> Show All -> Remove All IP Cache

- ❖ I use Python, yet I get “srun: error: ... Segmentation fault (core dumped)”

This is typical error, showing when the Conda environment gets corrupted. Create a new environment.

- ❖ Got an error like this: libstdc++.so.6: version `GLIBCXX_3.4.26' not found

If you already installed libgcc in your conda environment (like so: conda install libgcc), add the following line to your sbatch script right after the #SBATCH lines (replace 'username' and 'my_env' with yours):

```
export LD_LIBRARY_PATH=/home/username/.conda/envs/my_env/lib:$LD_LIBRARY_PATH
```

- ❖ Using java I get OOM errors. Java is automatically setting the MaxHeapSize of the JVM to 32GB while the code needs a job allocation of just 8GB.

Set java max heap size manually, to less than 8GB in order to fit the cluster allocated memory.